

Práctica 02 — Vectores 2D, Entrada Continua y Movimiento

Fundamentos de motores de videojuegos con C++ Facultad de Ciencias, UNAM

Luis Alberto Dacomba Méndez Usuario de GitHub: LuisAlberto99 **Repositorio:**

<https://github.com/LuisAlberto99/practica02-motor>

1. Demostración y cálculo matemático del error diagonal

Cuando el jugador presiona una sola tecla de dirección (por ejemplo, solo **D**), el vector de entrada es **(1, 0)**, cuya magnitud es:

$$||v|| = \sqrt{1^2 + 0^2} = 1$$

El desplazamiento en un segundo es entonces **1 × speed = 300 px/s**, que es la velocidad configurada.

Sin embargo cuando presiona **dos teclas al mismo tiempo** (por ejemplo **W** y **D**, para moverse en diagonal hacia arriba-derecha). El vector de entrada compuesto es **(1, -1)**, y su magnitud es:

$$||v_{diag}|| = \sqrt{1^2 + (-1)^2} = \sqrt{2} = 1.4142$$

Es decir, el vector diagonal es **41.4% más largo** que un vector de una sola dirección. Si este vector se usa directamente para mover al personaje, el desplazamiento real en diagonal termina siendo:

$$1.4142 \times 300 \text{ px/s} \approx 424.26 \text{ px/s}$$

en lugar de los 300 px/s configurados. Esto le da al jugador una ventaja de velocidad injusta al moverse en diagonal. **¿Cómo lo corregí?**

La solución es **normalizar** el vector de entrada antes de aplicar la velocidad. Un vector normalizado conserva la misma dirección pero su magnitud se ajusta a exactamente 1:

$$\hat{v} = v / ||v||$$

Aplicado al vector diagonal **(1, -1)**:

$$\hat{v}_{diag} = (1/\sqrt{2}, -1/\sqrt{2}) \approx (0.7071, -0.7071)$$

Comprobando su nueva magnitud:

$$||\hat{v}_{diag}|| = \sqrt{0.7071^2 + (-0.7071)^2} = \sqrt{0.5 + 0.5} = \sqrt{1} = 1$$

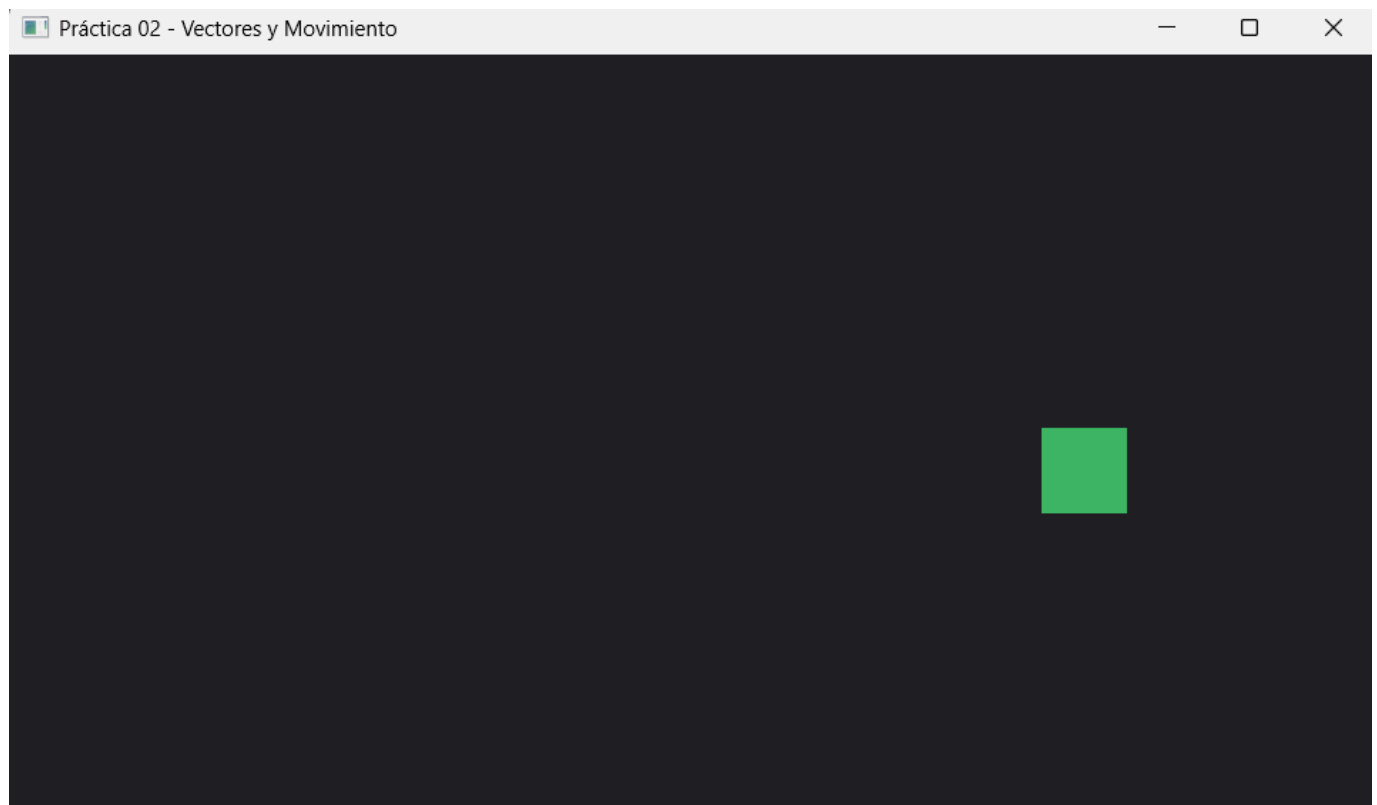
En el código, esto se implementó así dentro de `SDL_AppIterate` (o en `PhysicsUpdate` tras el paso de timestep fijo):

```
Vector2 input_dir{0.0f, 0.0f};
if (keys[SDL_SCANCODE_W] || keys[SDL_SCANCODE_UP])    input_dir.y -= 1.0f;
if (keys[SDL_SCANCODE_S] || keys[SDL_SCANCODE_DOWN]) input_dir.y += 1.0f;
if (keys[SDL_SCANCODE_A] || keys[SDL_SCANCODE_LEFT]) input_dir.x -= 1.0f;
if (keys[SDL_SCANCODE_D] || keys[SDL_SCANCODE_RIGHT]) input_dir.x += 1.0f;

if (input_dir.length_squared() > 0.0f)
{
    input_dir = input_dir.normalized();
}
```

Con esto, sin importar si el jugador se mueve en línea recta o en diagonal, la velocidad siempre es de 300 px/s.

2. Captura de pantalla — ventana en ejecución con el personaje en movimiento



3. Commit en GitHub

GitHub - practica02-motor/src/main.cpp at main · LuisAlberto99/practica02-motor

LuisAlberto99 / practica02-motor

Code Issues Pull requests Agents Actions Projects Wiki Security and quality More

practica02-motor Public

Pin Watch 0 Fork 0 Star 0

main

Go to file

Code

LuisAlberto99 Practica 2: Se implemento Vector2, ... b82d1a2 · 18 hours ago 1 Commit

build	Practica 2: Se implemento Vector2, n...	18 hours ago
src	Practica 2: Se implemento Vector2, n...	18 hours ago
CMakeLists.txt	Practica 2: Se implemento Vector2, n...	18 hours ago
Semana02.pdf	Practica 2: Se implemento Vector2, n...	18 hours ago

README

Add a README

Help people interested in this repository understand your project.

Add a README

About

No description, website, or topics provided.

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors 1

LuisAlberto99 Luis Dacomba

Languages

4. Ejercicio opcional — Clamping (confinamiento en pantalla)

Implementé la **Opción A: Confinar al personaje en la pantalla**. La idea es evitar que el rectángulo salga de los límites de la ventana (960×540 px).

Dentro de la función **PhysicsUpdate**, después de aplicar el desplazamiento a la posición del personaje, se agrego cuatro condiciones que "recortan" (clamp) la posición para que no sobrepase los bordes:

```
if (character.position.x < 0.0f)
{
    character.position.x = 0.0f;
}
if (character.position.x > 960.0f - character.size.x)
{
    character.position.x = 960.0f - character.size.x;
}
if (character.position.y < 0.0f)
{
    character.position.y = 0.0f;
}
```

```
if (character.position.y > 540.0f - character.size.y)
{
    character.position.y = 540.0f - character.size.y;
}
```

El límite izquierdo/superior es simplemente 0, y el límite derecho/inferior se calcula restando el tamaño del personaje al tamaño de la ventana, para que el rectángulo no se salga por el borde contrario (de lo contrario, solo su esquina superior izquierda quedaría dentro de la ventana, pero el resto del rectángulo se saldría).

Captura de pantalla del personaje confinado en el borde:

